

IPL Auction Platform – Advanced Architecture Design

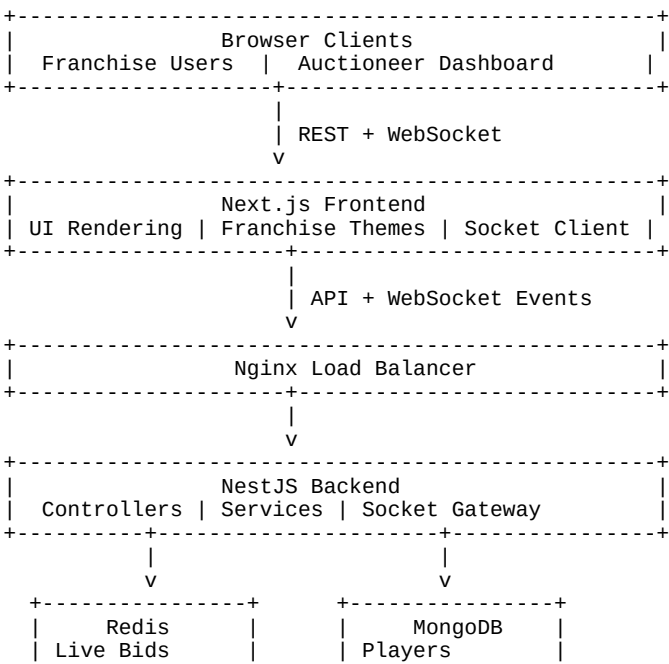
1. Project Goal

Design a scalable real-time auction platform inspired by the Indian Premier League (IPL) auction. The system must support multiple concurrent users representing franchises, live bidding updates, an auctioneer control panel, and reliable storage of auction results.

2. Technology Stack

Layer	Technology
Frontend	Next.js (React) + Socket.IO Client
Backend	NestJS (Node.js)
Realtime Communication	WebSockets via Socket.IO
Realtime Cache	Redis
Database	MongoDB
Authentication	JWT
Deployment	Docker
Reverse Proxy	Nginx
Scaling	Redis Adapter for Socket.IO

3. High-Level System Architecture



Auction Timer	Auction Results
Pub/Sub	User Data
+-----+	+-----+

4. Core Backend Services

Authentication Service

- Handles login and token validation

User Service

- Assigns random franchises to users
- Tracks user participation

Auction Service

- Controls auction state
- Validates bids
- Determines winners

Socket Gateway

- Handles real-time events
- Broadcasts bid updates

Player Service

- Manages player dataset
- Controls auction order

5. Real-Time Auction Event Flow

1. User joins lobby
2. Backend assigns a franchise
3. All users mark READY
4. Auctioneer starts the auction
5. Player details broadcast to all clients
6. Franchise users place bids
7. Backend validates bid
8. Redis updates highest bid
9. Socket event broadcasts new bid
10. Timer expires
11. Player sold
12. MongoDB stores final result

6. Scaling Strategy

To support many users simultaneously:

1. Use Redis Pub/Sub with Socket.IO Adapter
2. Run multiple NestJS backend instances
3. Use Nginx as a reverse proxy
4. Store only final results in MongoDB
5. Keep live state inside Redis

Benefits:

- Prevents race conditions
- Enables horizontal scaling
- Maintains fast real-time updates

7. Security Considerations

- JWT authentication
- Role-based access (Franchise / Auctioneer)
- Server-side bid validation
- Rate limiting to prevent spam bidding
- Secure WebSocket connections